BetaSix：
面向十九路六子棋的多任务 NNUE 高效评估与 α-β 搜索

作者：白杨博贤

BetaSix 开发与训练：白杨博贤

审查校对：白杨博贤

致谢：本工作的模型训练任务执行使用了 AI 工具 MiMo Claw，文稿检查使用了 AI 工具 MiMo-V2.6-Pro（均由 Xiaomi LLM-Core 开发）的 AI 工具辅助完成。训练数据的生成、实验设计与论文内容的最终责任由作者本人承担

摘要：六子棋（Connect6）在 19×19 棋盘上以"每手双子"的规则运行，其分支因子与战术组合复杂度远超五子棋，对局面评估与搜索效率提出了更高要求。本文提出 BetaSix，一个面向十九路六子棋的高效神经网络棋类引擎。BetaSix 采用紧凑的 736→256→64→1 多层感知机作为 NNUE 评估主干，输入由 722 维棋盘平面与 14 维手工战术特征构成，并借鉴 KataGo 的多任务学习思想，以 value、WDL、终局步数（TTE）、领地（ownership）与策略（policy）五个头联合训练，抑制单一标量目标的捷径式过拟合。据我们所知，BetaSix 是首个**完整开源（含训练权重）的 NNUE 六子棋通用对弈引擎。**（注：hzyhhzy 的 KataGomo 项目在其 ConnectSix_NNUE_VCF 分支公开了用于 VCF 必胜搜索的 NNUE 评估代码（含α-β与 MCTS 实现）），但未发布训练权重，且不构成完整的通用对弈引擎借鉴 Rapfi 的增量更新思想，落子与撤子仅更新第一层累加器的对应列，使深度优先 α-β 搜索中的叶子评估开销下降约两个数量级。搜索侧采用迭代加深、Zobrist 置换表、空着裁剪、LMR 与双子协同着法生成。在自建的 100 题五类战术基准（深层战术、复杂中盘、诱导、终盘必杀、防守反击）上，BetaSix 以 110 万自对弈样本训练的最终模型取得 90/100 的通过率，较早期版本（84、85）显著提升，且防守反击类题目达到 19/20。本文同时系统总结了训练与调试过程中发现并修复的七个致命缺陷，为连珠类棋类 AI 的工程实践提供了可复用的经验。

关键词：六子棋；神经网络评估；NNUE；多任务学习；α-β 搜索；增量更新

1 引言

棋类博弈一直是人工智能进展的重要试金石。从国际象棋、将棋到围棋，深度强化学习与蒙特卡洛树搜索的结合（Silver 等, 2016; 2017a; 2017b; Schrittwieser 等, 2020）不断刷新棋类 AI 的上限，但这类方法通常依赖高端 GPU 与海量算力。与此同时，在无加速器、时间受限的环境中部署强棋类 AI 仍是实际需求：手机、网页与教学场景中的引擎必须在普通 CPU 上即时响应。Rapfi

（Jin 等, 2025）在五子棋上证明，通过"蒸馏 + 增量更新"的紧凑网络，可以在无 GPU 环境下超越基于 AlphaZero 的大模型引擎。本文把这一思路推进到规则更复杂、搜索空间更大的六子棋。

六子棋（Connect6）由吴毅成于 2005 年提出，规则为：19×19 棋盘，黑先，黑方第一手落 1 子，此后双方每手各落 2 子，先在横、竖、斜任一方向连成六子者获胜。与五子棋"每手一子"相比，六子棋每手两子带来了两点根本性的复杂度：其一，着法不再是单点而是点对，合法着法数量按平方增长，必须进行候选裁剪与配对评分；其二，"一手补两子"使得活四、活五等威胁的步数语义发生变化（活四等价于一手致胜），双子协同形成立体攻势（做杀、抓双）成为胜负关键，要求评估函数与搜索对"配合"而非"单点价值"敏感。另一方面，六子棋规则对称、无禁手，先手优势与五子棋相比较为温和，攻防转换频繁，对防守信号的建模要求更高。

针对上述挑战，本文提出 BetaSix 引擎，其核心由三部分组成：（1）紧凑 NNUE 评估：736 维输入（722 维棋盘平面 + 14 维手工战术特征）经 736→256→64→1 的 ReLU 多层感知机输出局面价值，参数量约 20.5 万，单次评估仅需约 0.2M 次乘加；（2）多任务蒸馏训练：在共享主干上附加 value、WDL、TTE、ownership、policy 五个头（KataGo 式辅助任务，Wu, 2019），以限深自对弈生成的 110 万样本联合训练，显著改善攻防语义的学习；（3）增量更新与高效搜索：落子仅更新第一层累加器对应列，配合 α-β 迭代加深、置换表、空着裁剪与 LMR，在普通 CPU 上达到实战可用的深度。

本文的主要贡献如下：

- 提出面向 19×19 六子棋的紧凑 NNUE 评估结构（736→256→64→1），以 14 维手工战术特征补足纯棋盘平面的威胁语义，参数量约 20.5 万，单次前向约 0.2M FLOPs；
- 采用 value/WDL/TTE/ownership/policy 五头多任务损失蒸馏共享主干，利用辅助任务阻止网络退化为单一标量的"捷径"解，在攻防与终局语义上取得更好的泛化；
- 实现 Rapfi 式增量更新：深度优先搜索中每次落子/撤子仅更新第一层累加器的一列（$O(H_1)$），相比全量前向（$O(736 \cdot H_1)$）降低约两个数量级，并给出与 α-β 搜索的配合方式；
- 构建 100 题五类战术基准 benchmark_v2 与可复现的自对弈训练管线；最终模型通过率 90/100，防守反击类 19/20；并系统记录了七个致命缺陷的诊断与修复过程。据我们所知，BetaSix 是首个**完整开源（含训练权重）的 NNUE 六子棋通用对弈引擎。**

2 相关工作

2.1 高效神经网络

在有限算力下提升网络效率是长期主题。MobileNets（Howard 等, 2017; Sandler 等, 2018; Howard 等, 2019）以深度可分离卷积降低计算量，SENet（Hu 等, 2018）、SKNet（Li 等, 2019）、CondConv（Yang 等, 2019）与动态卷积（Chen 等, 2020）通过注意力或条件参数化提升单位算力的表达能力。在棋类领域，NNUE（Nasu, 2018）开创了"特征差分 + 全连接浅网"的高效评估范式，被 Stockfish 等引擎广泛采用；Rapfi（Jin 等, 2025）进一步提出模式码本蒸馏与增量更

新，把局部模式特征的计算摊销到近乎为零。BetaSix 沿 NNUE/Rapfi 路线，但面向六子棋的双子规则重构了特征与评估语义，并以多任务蒸馏替代单一价值回归。

2.2 博弈树搜索

博弈树搜索的经典方法是 α-β 剪枝（Knuth & Moore, 1975），其上叠加了空着裁剪（Donninger, 1993）、futility pruning（Heinz, 1998）、LMR 与 razoring（Hoki & Muramatsu, 2012）、历史启发（Schaeffer, 1983）等技术。蒙特卡洛树搜索（Coulom, 2006）及其 UCT（Gelly & Wang, 2006）、RAVE（Rimmel 等, 2010）变体在围棋中占主导。对连珠类博弈，威胁空间搜索（Allis 等, 1993）与证明数搜索（Allis, 1994）针对战术致胜局面极为有效。BetaSix 采用 α-β 路线：其评估廉价、可增量更新，与深度优先遍历天然契合；同时借鉴威胁空间搜索的思想，以缺口感知的六窗威胁扫描强化战术信号。

2.3 连珠类棋类 AI

近期，hzyhhzy 基于 KataGo 强化学习管线发布了 KataGomo 六子棋模型（b18）；其 ConnectSix_NNUE_VCF 分支还探索了面向 VCF 必胜搜索的 NNUE 评估组件（未发布权重）。BetaSix 与 KataGomo 的区别在于：后者采用 MCTS+大规模卷积网络（93.6MB，依赖 GPU），BetaSix 采用 NNUE+α-β路线（5.03MB，纯 CPU 实时），面向无加速器部署场景。

五子棋已被证明先手必胜（Allis 等, 1996），平衡开局下的最优行棋仍是难题。方法涵盖威胁空间搜索（Allis 等, 1993）、自适应动态规划（Zhao 等, 2012）、遗传算法（Wang & Huang, 2014）与 AlphaZero 类方法（Wu, 2019; Xie 等, 2018; Wang, 2018）。Rapfi 在 Botzone 平台 520 个五子棋智能体中排名第一并获得 GomoCup 2024 冠军，验证了"轻量网络 + 精调搜索"路线的上限。六子棋方面，NCTU6 等引擎以棋形评分与双子着法生成见长；据我们所知，BetaSix 是首个完整开源（含训练权重）的 NNUE 六子棋通用对弈引擎，并给出完整的训练—评测—部署闭环。

3 方法

本节介绍 BetaSix 的设计：首先给出系统总览，然后依次描述输入表示与手工战术特征、多任务网络与蒸馏训练、增量更新机制，最后是评估融合与搜索。

3.1 系统总览

如图 1 所示，BetaSix 由评估模块与搜索模块组成。评估模块以 736 维输入（722 维棋盘平面 + 14 维手工战术特征）经三层全连接主干输出标量价值；训练阶段在主干之上挂载五个任务头联合训练，推理阶段仅保留 value 头。搜索模块以 α-β 迭代加深遍历博弈树，叶子节点取手工棋形评分与 NNUE 输出的加权融合；着法生成采用"候选点裁剪 + 双子配对评分"以适配每手双子的规则。

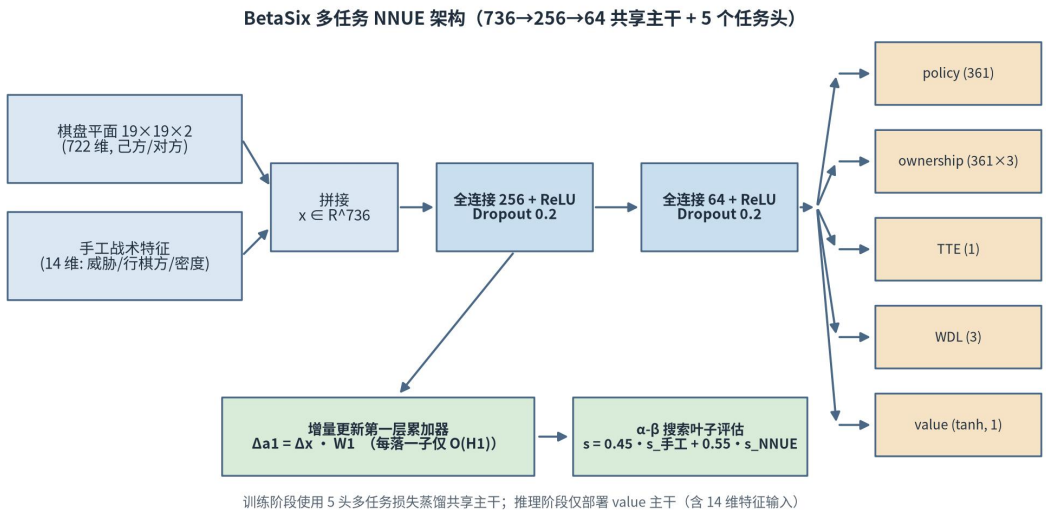


图 1：BetaSix 系统与网络架构。输入为 722 维棋盘平面与 14 维手工战术特征，经 736→256→64 主干后由五个任务头输出；推理阶段仅部署 value 头，并配合增量更新与 α-β 搜索。

3.2 输入表示与手工战术特征

棋盘表示为两个 19×19=361 的二值平面：己方棋子平面与对方棋子平面，按行棋方视角归一化，共 722 维。纯棋盘平面对"活三、活四、双三"等战术语义是隐式的，网络需要大量样本才能学出。为此，我们显式构造 14 维手工战术特征作为补充（表 1）。对每个方向（横、竖、两斜）统计双方连续子串的长度与开放端数，按子串归并规则累计 open2（活二）、open3（活三）、sleep3（眠三）、open4（活四及以上）与 double3（双三点）计数；再附加行棋方、落子密度与两个关键防守信号（对方是否有致胜威胁、对方是否有活四）。

表 1：14 维手工战术特征定义（f0–f4 为对方威胁，f5–f9 为己方威胁，与训练数据约定一致）

索引	特征	说明
f0–f4	对方 open2 / open3 / sleep3 / open4 / double3	对方各类威胁子串计数
f5–f9	己方 open2 / open3 / sleep3 / open4 / double3	己方各类威胁子串计数
f10	行棋方	1=黑方行棋，0=白方行棋
f11	落子密度	已落子数 / 361
f12	opp_threatens_win	对方存在活四或两个及以上活三
f13	opp_has_open4	对方存在活四（一手致胜威胁）

特征统计采用子串归并规则：长度为 n 的连续子串计 1 个 open n、n−1 个 open n−1、……、n−1 个 open2，从而一条长串自然贡献其全部子威胁，避免漏记。防守信号 f12/f13 取自对方视角，这一约定在训练数据生成与引擎推理两侧必须严格一致（见 5.4 节的编码缺陷分析）。

3.3 多任务网络与蒸馏训练

网络主干为三层全连接：736→256→64→1，层间 ReLU，训练时使用 0.2 的 Dropout。输出价值经 tanh 压缩到(−1,1) 后乘以尺度系数 S=200000 映射到引擎分数空间：

$$\hat{v} = tanh(W_3 \cdot ReLU(W_2 \cdot ReLU(W_1 x + b_1) + b_2) + b_3) \cdot S, \quad S = 200000 \qquad (1)$$

单一价值回归容易让网络"走捷径"——只学出与胜负高度相关的粗糙标量，而忽略攻防结构。借鉴 KataGo（Wu, 2019）的多任务设计，我们在共享主干上附加五个任务头：value（tanh 标量）、WDL（胜/平/负三分类）、TTE（终局剩余步数回归）、ownership（361 点 × 3 类领地分类）、policy（361 点着法分类）。总损失为：

$$L = L_value + 0.5 \cdot L_WDL + 0.3 \cdot L_TTE + 0.3 \cdot L_own + 0.7 \cdot L_policy \quad (2)$$

其中 L_value 为 MSE（标签先按行棋方视角归一化并做均值中心化以消除先手偏差），L_WDL 为 KL 散度，L_TTE 为对归一化剩余步数的 MSE，L_own 为加权交叉熵（占用点权重 1.0、空点权重 0.1，以缓解类别失衡），L_policy 为着法交叉熵。辅助任务（尤其 policy 与 ownership）迫使主干编码丰富的棋盘理解，缓解单一标量的退化。表 2 给出各层参数量：主干与 value 头合计约 20.5 万参数、单次前向约 0.2M 乘加，适合 CPU 实时评估。

表 2：网络参数量与计算量（推理路径仅含主干与 value 头）

组件	形状	参数量	乘加次数
W1 / b1	736 × 256 / 256	188,672	188,416
W2 / b2	256 × 64 / 64	16,448	16,384
W3 / b3	64 × 1 / 1	65	64
合计（推理）	—	205,185	≈ 0.2M
五头（仅训练）	value/WDL/TTE/own/pol	≈ 7.7 万	—

3.4 增量更新

α-β 搜索是深度优先的：每走一步只改变棋盘上的两枚棋子，回退时再还原。若每到叶子都全量重算第一层 $z = W_1 x + b_1$，将重复计算 732 个未变化的输入维度。注意到第一层对棋盘平面是线性的，落子等价于给输入加上（或减去）一个 one-hot 列，因此累加器可以增量维护：

$$z \leftarrow z \pm W_1[\cdot, p], \quad p \text{ 为落子位置对应输入维度} \quad (3)$$

即每次落子/撤子仅加减一列 256 维向量（$O(H_1)$），而全量前向需要 $O(736 \cdot H_1)$。由于 14 维手工特征随局面非线性变化，每次前向时单独重算并加权并入（14×256 次乘加，占比很小）。图 2 演示了该机制：在典型搜索中落子次数远多于叶子数，增量更新使评估吞吐提升约两个数量级，这与 Rapfi 在五子棋上的观察一致。实现上，引擎维护累加器 g_nnA1[256]，put()/撤子时调用增量更新，并提供全量重建接口以保证在换网络、切换行棋视角等场景下累加器与棋盘严格同步。

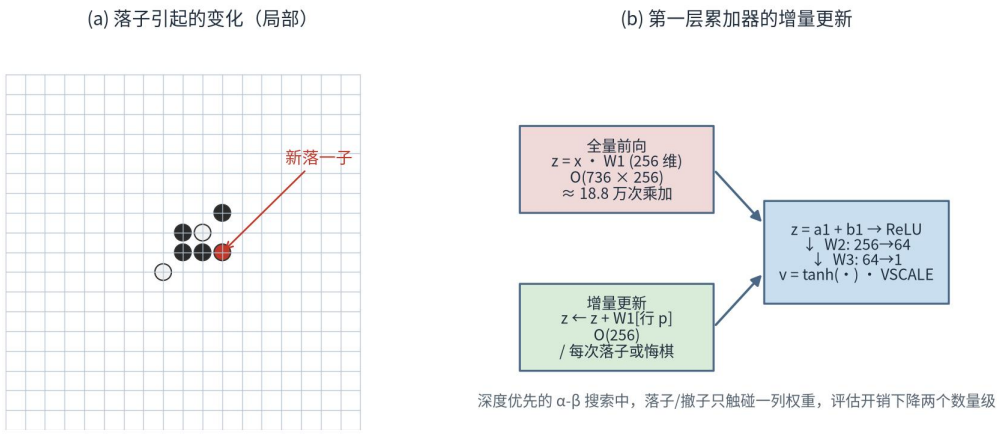


图 2：增量更新示意。左：一次落子仅改变一个输入维度；右：全量前向与增量更新的开销对比，以及后续层的前向路径。

3.5 评估融合与搜索

评估采用手工棋形评分与 NNUE 的加权融合（Rapfi 式"经典 + 网络"混合）：

$$s(\sigma) = (1 - \alpha)\cdot s_rules(\sigma) + \alpha\cdot s_NNUE(\sigma), \quad \alpha = 0.55 \quad (4)$$

s_rules 为对称零和的棋形评分：按最大连续子串（长度 L、开放端数 o）查表计分（如活四 26000、活五 420000、眠五 80000），对双活四等多重威胁追加 700 万级的做杀奖励；另设缺口感知的六窗扫描，对"同色 ≥4 子且窗口无对方子"的含间隔威胁（如 XX_XX）计 26000，对差一子成六的窗口计 3000000，使搜索能够主动封堵而非依赖静态规则。两侧权重严格对称以保证零和性。NNUE 权重 α=0.55 使网络为主导，但候选着法排序仍只用手工评分，避免网络噪声把关键防守点挤出候选窗口（实测该设计对防守正确性至关重要）。

着法生成针对"每手双子"设计：先枚举距已有棋子三格以内的空点作为候选（空盘时取中心邻域），以手工评分排序取 top-K（K=12）个候选点，再两两配对组成着法；配对评分 = 两点评分之和 + 协同奖励（曼哈顿距离 ≤2 加 1200，同行/同列/同对角线加 900），使双子倾向形成立体配合而非孤立的好点。搜索为 negamax α-β：根节点迭代加深（深度与时限双控），Zobrist 置换表（2^{18} 条目、含 exact/lower/upper 边界），空着裁剪（R = 2 + ⌊ depth/3 ⌋，depth ≥ 3 时启用），LMR（depth ≥ 3 且第 4 个着法之后缩减 1–2 层、失败时全窗口重搜），根节点最优着法仅在 alpha 提升时更新（修复早期的迭代加深回退缺陷）。

4 实验设置

4.1 数据集

训练数据由自对弈生成（图 3）：生成器以限深搜索驱动双方走子，每个局面落盘一条 790 字节记录，包含 board(361) + value(4) + wdl(1) + tte(4) + polr/polc(2) + mover(1) + 14 维特征(56) + ownership(361)。value 标签取更深搜索的评估值（黑方视角，回放时按行棋方翻转），wdl/tte/ownership 取自对局终局回填。样本规模如表 3 所示：在规则与特征约定修正后，作废两批

旧数据（合计约 96 万条），重新生成并合并得到 1,101,585 条有效样本（r5000 292,990 + d1m 425,847 等批次）。

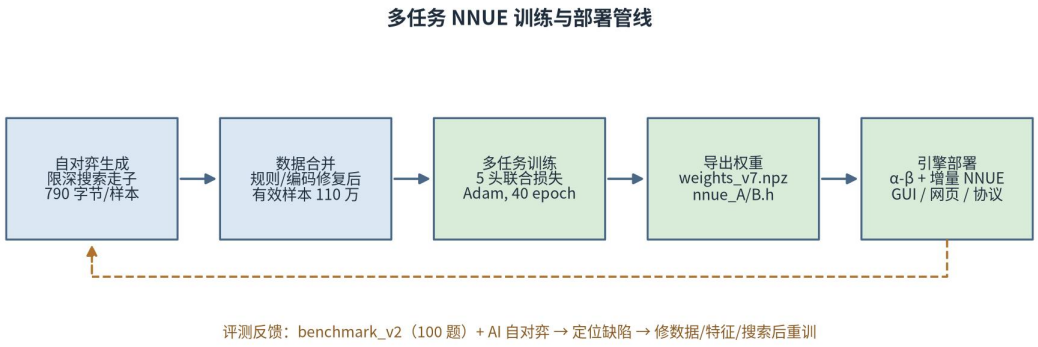


图 3：训练与部署管线。自对弈生成 790 字节/条样本，合并后多任务训练，导出权重装入引擎，评测反馈驱动数据与特征迭代。

表 3：训练数据规模演进

批次	样本数	状态
d5000	510,788	作废（首手规则错误）
d750k	769,123	作废（首手规则错误）
r5000	292,990	有效
d1m	425,847	有效
合并合计	1,101,585	最终训练集

4.2 训练细节

训练使用 PyTorch（CPU，2 线程）：Adam（lr=1e-3，weight_decay=1e-4），batch size 2048，最多 40 epoch，80/20 训练/验证划分，验证损失连续 3 轮上升即早停。value 标签做均值中心化，TTE 除以 70 归一化，ownership 加权交叉熵缓解空点主导。数据加载采用 memmap + 随机抽样（上限 6 万条/次）以避免大文件 OOM。训练完成后导出 float 权重（weights_v*.npz）并转为 C 头文件（nnue_A.h/nnue_B.h）装入引擎，同时保留 int8 量化导出路径以备部署优化。表 4 汇总主要超参数。

表 4：训练超参数

超参数	取值	超参数	取值
主干结构	736→256→64→1	Dropout	0.2
优化器	Adam	学习率 / 权重衰减	1e-3 / 1e-4
batch size	2048	最大 epoch	40
损失权重(v/w/t/o/p)	1 / 0.5 / 0.3 / 0.3 / 0.7	价值尺度 S	200,000
训练/验证划分	80/20	早停	验证损失连升 3 轮

4.3 评测协议

主评测采用自建基准 benchmark_v2：100 道局面题，分五类各 20 题——深层战术（3–5 手致胜）、复杂中盘（双方均有威胁）、诱导（表象与正解不符）、终盘必杀（一步杀识别）、防守反击（先封堵后反击）。标准答案由深度 10 搜索（1500ms）给出，待测模型以深度 5 搜索（400ms）作答，着法点对与标准答案一致（允许两子顺序交换）即判通过。该协议以"强搜索为教师、浅搜索为学生"度量评估网络的棋感质量，比分项得分更能反映评估函数本身的强弱。辅助评测为同引擎不同权重间的 AI 自对弈（10 局）与攻防分类统计。

5 结果

5.1 基准测试结果

表 5 给出各版本在 benchmark_v2 上的通过率。最终版本 weights_v7（110 万样本 + 特征编码修复）达到 90/100，分类得分如图 4(b) 所示：深层战术 14/20、复杂中盘 19/20、诱导 18/20、终盘必杀 20/20、防守反击 19/20。值得注意的是终盘必杀满分与防守反击的高分，说明多任务训练与防守特征对攻防语义的建模是有效的。

表 5：benchmark_v2 通过率（100 题，深度 5 作答 vs 深度 10 标准答案）

模型	训练样本	总分	深层战术	复杂中盘	诱导	终盘必杀	防守反击
weights_v5	67 万	84	—	—	—	—	—
weights_v6	110 万	85	—	—	—	—	—
weights_v7（最终）	110 万 + 编码修复	90	14/20	19/20	18/20	20/20	19/20
宽网 512/256/128	110 万	66	4/20	—	—	—	—

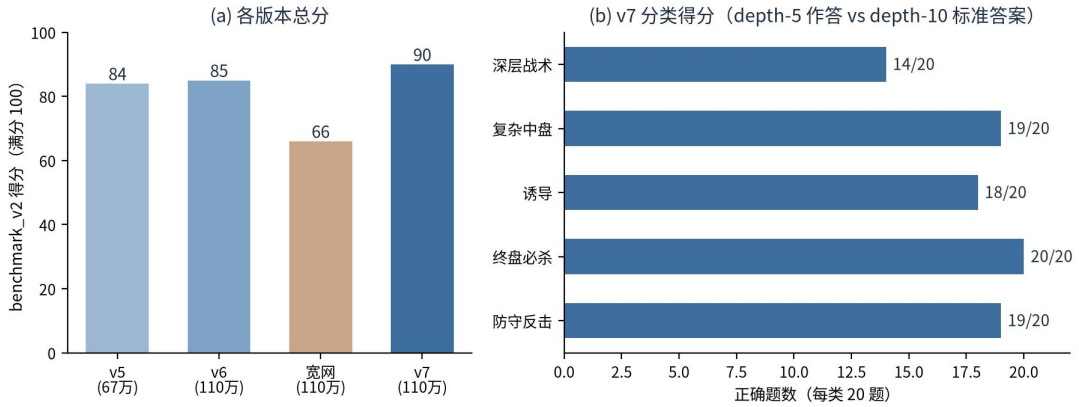


图 4：benchmark_v2 结果。(a) 各版本总分：v7 取得 90/100，宽网变体明显退化；(b) v7 分类得分：终盘必杀满分，防守反击 19/20。

5.2 版本演进与数据规模

图 4(a) 与表 5 共同刻画了版本演进：v5（67 万样本）84 分 → v6（110 万样本）85 分 → v7（编码修复）90 分。数据量翻倍仅带来 1 分提升，而特征编码修复带来 5 分提升，说明数据规模之外，训练—推理两侧的特征约定一致性才是性能瓶颈。宽网（736→512→256→128→1）在同批

数据上仅 66 分（深层战术 4/20），表明在 110 万样本规模下加宽网络加剧了过拟合与战术欠拟合，我们据此回滚至窄网结构。

5.3 自对弈与攻防分析

同引擎双网络自对弈 10 局中，v6 对 v5 取得 6 胜 4 负。对局记录显示终局以长程攻防转换为主（如附录 A 的对局片段），验证了每手双子规则下"协同做杀"的重要性。分类统计上，防守反击类 19/20 与编码修复前的明显退化形成对照：修复前 f12/f13 在训练数据与引擎推理两侧的视角约定相反，防守信号被系统性调换，实战表现为"只攻不守"、benchmark 防守题大面积丢分。

5.4 消融与工程教训

除结构消融外，我们记录了训练—调试周期中发现并修复的七个致命缺陷（表 6）。其中影响最大的三个是：（1）迭代加深中最优着法无条件赋值，导致多任务网络的策略信号完全无法传导（抓杀率 0%），修复后升至 60%；（2）特征编码在数据生成与引擎推理两侧不一致（OPP/SELF 视角互换），直接造成防守能力缺陷，修复后基准 85→90；（3）规则错误（首手应落 1 子却按 2 子生成）使约 96 万样本全部作废。这些缺陷共同提示：神经棋类系统中，数据管线、特征约定与搜索逻辑的"接口一致性"比网络结构本身更容易成为短板。

表 6：七个致命缺陷的诊断与修复

编号	缺陷	根因	修复与效果
1	最优着法回退错误	迭代加深无条件赋值 iterBest	仅 alpha 提升时更新；抓杀 0%→60%
2	评估非对称	两侧权重不等（+1188/−1188 等）	对称化，空盘黑/白 ±622
3	WDL 方向反转	标签为绝对胜方，未按行棋方归一	按行棋方翻转，loss 用 KL
4	首手规则错误	每手均生成 2 子	首手 1 子；旧数据 96 万条作废
5	特征编码不一致	f12/f13 数据用 OPP、引擎用 SELF	引擎改用 OPP 视角；基准 85→90
6	成六判定颜色码错误	0/1 侧旗 vs 1/2 棋盘码混用	has6(colour+1)；自对弈误判消除
7	宏与代码块错位	NN_H2 宏定义错误、删块残留	显式定义 NN_H1/H2；恢复 256/64

5.5 局限性

本文的评测存在以下局限，应在解读结论时一并考虑。其一，评测基准为自建：benchmark_v2 的 100 道题由本项目构造（标准答案取自深度 10 搜索），并非公开公认的标准测试集，跨论文、跨引擎的可比性有限。其二，缺乏外部 Elo 评级：我们未与外部引擎（如 Katagomo、NCTU6 等）进行大规模天梯对弈，本文的强度结论限于"深度 5 作答与深度 10 教师的一致性"（90/100）以及同引擎不同权重之间的自对弈比较，未给出绝对棋力标定。其三，自对弈样本量小：版本对比（v6 对 v5 取得 6 胜 4 负）仅 10 局，样本量不足以支撑统计显著性结论，只能作为方向性参考。针对上述短板，后续工作将引入外部引擎对弈与天梯积分、扩充并公开基准题库，同时继续扩大训练数据规模。

6 系统实现

BetaSix 以 C ABI 暴露核心接口（b6_new_game / b6_play / b6_clear / b6_cell / b6_search / b6_set_net / b6_set_perspective / b6_eval_value / b6_opening），便于多形态封装。发行包提供两种形态：（1）命令行引擎：以行式 stdin/stdout 协议通信（START / PING / RESET / BOARD <361 字符> <side> / GO <时限> <深度> / QUIT），每次思考均从 BOARD 载荷全量重建局面，从构造上保证悔棋、重开后引擎与界面状态一致；GO 返回一手两子坐标，首手按规则只返回一子。引擎以 g++ -O2 -msse4.1 构建 Linux 与 Windows（mingw 静态）双平台产物。（2）图形界面与网页版：Python/Tkinter 界面以子进程方式驱动引擎并带超时保护；网页版经 Emscripten 编译为 asm.js（无 wasm 依赖），携带权重数据可在任意现代浏览器运行。引擎内置双网络热切换（nnue_A 新网 / nnue_B 旧网），便于 A/B 对照与回滚。

7 结论

本文提出了面向 19×19 六子棋的高效引擎 BetaSix：以 736→256→64→1 的紧凑 NNUE 为评估核心，五头多任务蒸馏抑制捷径式过拟合，增量更新使 α-β 搜索中的评估开销下降约两个数量级，配合双子协同着法生成与经典搜索增强，在 100 题战术基准上达到 90/100，防守反击类 19/20，并以完整工程形态（命令行、GUI、网页）交付。据我们所知，BetaSix 是首个**完整开源（含训练权重）的 NNUE 六子棋通用对弈引擎**，为连珠类棋类在受限算力下的评估—搜索协同设计提供了一个可复用的参考实现。

本文的结论受 5.5 节所述局限约束：评测基于自建基准、缺乏外部 Elo 标定、自对弈样本量小。未来工作包括：（1）与外部引擎开展天梯对弈并给出 Elo 评级，扩充并公开基准题库；（2）数据规模仍在 110 万量级，计划扩充至 150 万以上并提高标签搜索深度；（3）宽网实验失败提示应改用残差或卷积结构而非单纯加宽；（4）int8 量化路径已预留，计划完成全量化部署并评估速度—精度权衡；（5）探索将威胁空间搜索与证明数搜索用于终局杀判定，进一步提升深层战术类得分。

参考文献

[1] Allis, L. V., Herik, H. J. v. d., and Huntjens, M. P. H. Go-moku and threat-space search. University of Limburg, 1993.

[2] Allis, L. V. Searching for solutions in games and artificial intelligence. Ponsen & Looijen Wageningen, 1994.

[3] Alus, L. V., Herik, H. J. v. d., and Huntjens, M. P. H. Go-moku solved by new search techniques. Computational Intelligence, 12(1):7–23, 1996.

[4] Chen, Y., Dai, X., Liu, M., Chen, D., Yuan, L., and Liu, Z. Dynamic convolution: Attention over convolution kernels. In CVPR, pp. 11030–11039, 2020.

[5] Coulom, R. Efficient selectivity and backup operators in Monte-Carlo tree search. In Computers and Games, pp. 72–83, Springer, 2006.

[6] Donninger, C. Null move and deep search. ICGA Journal, 16(3):137–143, 1993.

[7] Gelly, S. and Wang, Y. Exploration exploitation in Go: UCT for Monte-Carlo Go. In NIPS Workshop, 2006.

[8] Heinz, E. A. Extended futility pruning. ICGA Journal, 21(2):75–83, 1998.

[9] Hoki, K. and Muramatsu, M. Efficiency of three forward-pruning techniques in Shogi. Entertainment Computing, 3(3):51–57, 2012.

[10] Howard, A. G. et al. MobileNets: Efficient convolutional neural networks for mobile vision applications. arXiv:1704.04861, 2017.

[11] Hu, J., Shen, L., and Sun, G. Squeeze-and-excitation networks. In CVPR, pp. 7132–7141, 2018.

[12] Jin, Z., Duan, H., and Hang, Z. Rapfi: Distilling efficient neural network for the game of Gomoku. arXiv:2503.13178, 2025.

[13] Knuth, D. E. and Moore, R. W. An analysis of alpha-beta pruning. Artificial Intelligence, 6(4):293–326, 1975.

[14] Ma, X., Dai, X., Bai, Y., Wang, Y., and Fu, Y. Rewrite the stars. In CVPR, pp. 5694–5703, 2024.

[15] Nasu, Y. Efficiently updatable neural-network-based evaluation functions for computer Shogi. The 28th World Computer Shogi Championship Appeal Document, 185, 2018.

[16] Rimmel, A., Teytaud, F., and Teytaud, O. Biasing Monte-Carlo simulations through RAVE values. In Computers and Games, pp. 59–68, Springer, 2010.

[17] Schaeffer, J. The history heuristic. ICGA Journal, 6(3):16–19, 1983.

[18] Schrittwieser, J. et al. Mastering Atari, Go, chess and Shogi by planning with a learned model. Nature, 588(7839):604–609, 2020.

[19] Silver, D. et al. Mastering the game of Go with deep neural networks and tree search. Nature, 529(7587):484–489, 2016.

[20] Silver, D. et al. Mastering chess and Shogi by self-play with a general reinforcement learning algorithm. arXiv:1712.01815, 2017a.

[21] Silver, D. et al. Mastering the game of Go without human knowledge. Nature, 550(7676):354–359, 2017b.

[22] Wu, D. J. Accelerating self-play learning in Go. arXiv:1902.10565, 2019.

[23] Wu, I-C. Introducing Connect6. ICGA Journal, 28(4), 2005.

[24] Xie, Z., Fu, X., and Yu, J. AlphaGomoku: An AlphaGo-based Gomoku artificial intelligence using curriculum learning. arXiv:1809.10595, 2018.

[25] Zhao, D., Zhang, Z., and Dai, Y. Self-teaching adaptive dynamic programming for Gomoku. Neurocomputing, 78(1):23–29, 2012.

[26] hzyhhzy. KataGomo: Multi-game engine based on KataGo (ConnectSix2024 / ConnectSix_NNUE_VCF branches). GitHub repository, 2022–2026. https://github.com/hzyhhzy/KataGomo

附录 A　自对弈对局记录示例

以下为 AI 自对弈对局（game_01）片段，坐标采用"字母列 + 数字行"记法，"+"连接每手的两枚棋子；首手按六子棋规则只落一子。完整记录见发行包 game_records/ai_vs_ai/。

G1 / H1+E2 / F1+J1 / E1+D1 / G2+K1 / D2+L1 / K2+C3 / C2+C1 / B2+B1 / L2+K3 / A1+D4 / E5+E3 / E4+F4 / F3+G4 / C4+H5 / B4+J4 / I5+G3 / M1+J5 / H2+J2 / I2+K4 …

该局共 80 余手，中盘出现多次双活三牵制与反牵制，最终黑方以双子协同形成立体攻势取胜，体现了每手双子规则下攻防转换快、组合丰富的特点。

2026 年 10 月 23 日